Adversarial Robustness and Enhanced Detection for Parcel Tampering

Team: **Group 7**

Team Members: Nishith Mohan Mittal, Rakesh Sharma, Pallav Krishna Rakesh, Yashwanth Maringanti

Team Emails: nishith@arizona.edu , rakeshs@arizona.edu , pallav12364@arizona.edu , ymaringanti@arizona.edu

GitHub: https://github.com/r-sharma/tampar

1. Abstract:

The security of parcel logistics is increasingly threatened by sophisticated tampering attempts that can evade automated visual inspection systems. This project extends TAMPAR, a state-of-the-art visual tampering detection system for parcel logistics by addressing two critical limitations: vulnerability to adversarial attacks and suboptimal feature discrimination. Using approximately 1,300 high-resolution parcel images from the TAMPAR dataset, we implement FGSM and PGD adversarial attacks that reduce baseline detection accuracy from ~82% to ~72%, exposing a significant security gap. To counter this, we fine-tune the SimSAC change detection model using a novel direct change map loss objective that directly optimises the model's detection signal rather than an auxiliary embedding head. Combined with XGBoost classification over the fine-tuned similarity features, our enhanced pipeline recovers adversarial accuracy to 85.99%, a ~13 percentage point improvement, while raising ROC-AUC from 0.77 to 0.94 and recall from 59% to 95% under attack conditions. Performance on clean images also improves, from 83.85% to 87.00%. This work establishes the first comprehensive adversarial benchmark for parcel tampering systems and demonstrates a practical defence pipeline applicable to last-mile delivery, postal supply chains, and logistics security operations.

2. Introduction:

Physical parcel tampering, the unauthorised opening, resealing, or contents replacement of packages in transit, poses a growing threat to e-commerce, pharmaceutical distribution, and sensitive document delivery. In last-mile logistics, where millions of parcels change hands daily, manual inspection is neither scalable nor consistent. Automated vision-based systems have emerged as a promising solution, with recent work such as TAMPAR establishing a benchmark for detecting surface-level tampering by comparing pre-shipment reference images against delivery images.

However, the real-world deployment of such systems faces a fundamental security risk: adversarial attacks. A malicious actor, such as a delivery employee seeking to conceal tampering, could apply imperceptible digital perturbations to images captured in the logistics pipeline, causing the detection model to misclassify a tampered parcel as intact. This class of attacks, well-studied in other computer vision domains, has received no systematic evaluation in the context of parcel tampering detection.

Beyond adversarial robustness, existing tampering detection systems rely on generic similarity metrics and simple threshold-based classifiers that were not designed for the fine-grained discrimination required to distinguish subtle surface modifications. As parcel volumes and tampering methods grow more sophisticated, both resilience and precision become operational requirements.

This project addresses these gaps by evaluating the adversarial vulnerability of the TAMPAR pipeline, developing a fine-tuned detection model robust to such attacks, and replacing the baseline

classifier with a more powerful machine learning approach, advancing the system towards practical, reliable deployment in real-world logistics environments.

3. Objective: This project aims to enhance the security and robustness of the TAMPAR (Tampering Detection for Parcel Logistics) system through two primary extensions:

A. Conducting comprehensive **adversarial attack analysis** to evaluate and improve the model's resilience against digital perturbations (FGSM and PGD attacks), and implementing adversarial training to create a robust variant; and

B. Implement parcel-specific contrastive learning to fine-tune SimSaC for enhanced feature representations, and implementing **Random Forest, XGBoost** and ensemble classifiers (Random Forest and XGBoost) to compare their performance with tampering classification accuracy.

4. Data:

Source: TAMPAR dataset from Zenodo (https://zenodo.org/records/10057090)**[5]**

Reference: Naumann et al., "TAMPAR: Visual Tampering Detection for Parcels," WACV 2024

Dataset Size: The dataset comprises approximately 1,300 high-resolution images (4032×3024 pixels) containing 30 unique parcel IDs distributed across train, validation, and test splits. Each image includes comprehensive annotations: bounding boxes, segmentation masks, 8 key-points per parcel, and binary tampering labels. The data consists of real-world parcel images captured in logistics environments, formatted according to COCO annotation standards.

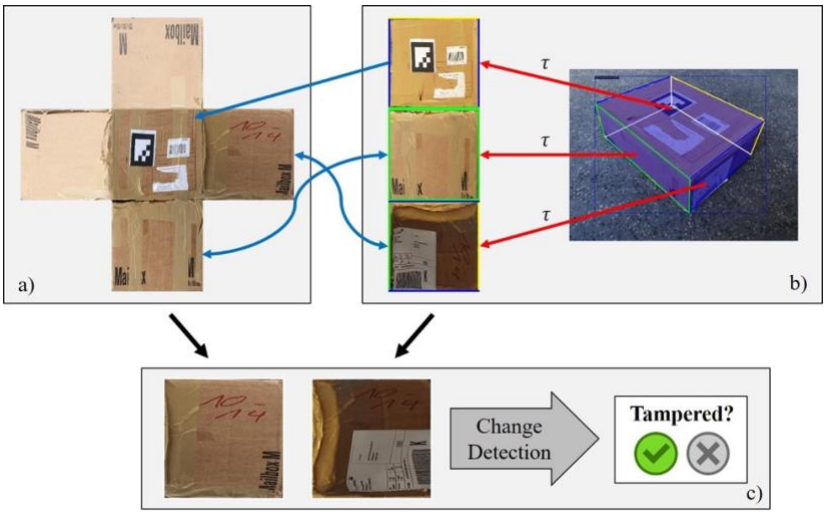

Figure 1: The base detects tampering by comparing the full parcel texture from a database (a) with the viewpoint-invariant parcel side surfaces of a single image by exploiting parcel corner point predictions (b). Appearance change detection is performed for each pair of matching parcel side surfaces to identify tampering (c). © IEEE 2024.

5. Literature Review:

Visual inspection systems for detecting physical tampering in logistics environments represent an emerging application of computer vision. Naeem et al. [1] introduced TAMPAR, a benchmark dataset and baseline approach for identifying surface damage on parcels using keypoint-normalized comparison between pre-shipment and delivery images. While their work established foundational methods for this domain, two significant limitations remain unaddressed: vulnerability to adversarial manipulation and suboptimal feature representations for subtle tampering detection.

Adversarial Robustness in Visual Systems
Deep learning models are susceptible to adversarial perturbations, carefully crafted inputs designed to cause misclassification while remaining imperceptible to humans. Madry et al. [2] formalized adversarial training using Projected Gradient Descent (PGD) as a principled defense mechanism, demonstrating improved robustness on standard benchmarks. Subsequent work has shown that adversarial vulnerabilities extend beyond digital perturbations to physical-world attacks; Thys et al. [4] successfully demonstrated adversarial patches that fool person detection systems in real surveillance scenarios. These findings raise concerns for logistics applications where malicious actors such as delivery personnel seeking to conceal tampering could exploit such vulnerabilities. However, adversarial robustness has not been evaluated for tampering detection pipelines like TAMPAR, leaving a critical gap in understanding system reliability under attack conditions.

Representation Learning for Fine-Grained Visual Discrimination
Detecting subtle surface modifications requires discriminative feature representations. Contrastive learning approaches, particularly SimCLR [3], have demonstrated that self-supervised pretraining produces features that generalize effectively across downstream visual tasks, often surpassing supervised alternatives. These representations capture semantic similarities that may prove beneficial for distinguishing tampered from intact parcel surfaces. Yet, domain-specific adaptation of contrastive learning to logistics imagery, combined with advanced classification approaches such as ensemble methods, remain unexplored.

Research Gap and Our Contribution
This project extends TAMPAR along two axes. First, we evaluate and improve the system's robustness against FGSM and PGD adversarial attacks through adversarial training. Second, we investigate whether contrastive fine-tuning combined with ensemble classifiers (Random Forest, XGBoost) can improve detection accuracy for subtle tampering cases. Together, these extensions address operational requirements for last-mile delivery where both adversarial manipulation and high false-positive rates pose practical challenges.

6. Methods:

Pipeline Overview:

This project enhances the TAMPAR system by adding four more stages to the original pipeline: (1) adversarial attack generation, (2) adversarial training for robust keypoint detection, (3) SimSaC fine-tuning through parcel-specific contrastive learning, and (4) comparison with enhanced classification methods (Random Forest, XGBoost and their ensemble).

The proposed approach adds adversarial robustness (Extension A) and improved tampering detection capabilities (Extension B) to the existing architecture. There are in total 7 steps in the pipeline architecture (Figure 2). Three of those steps are unchanged and four steps have been added as extensions.

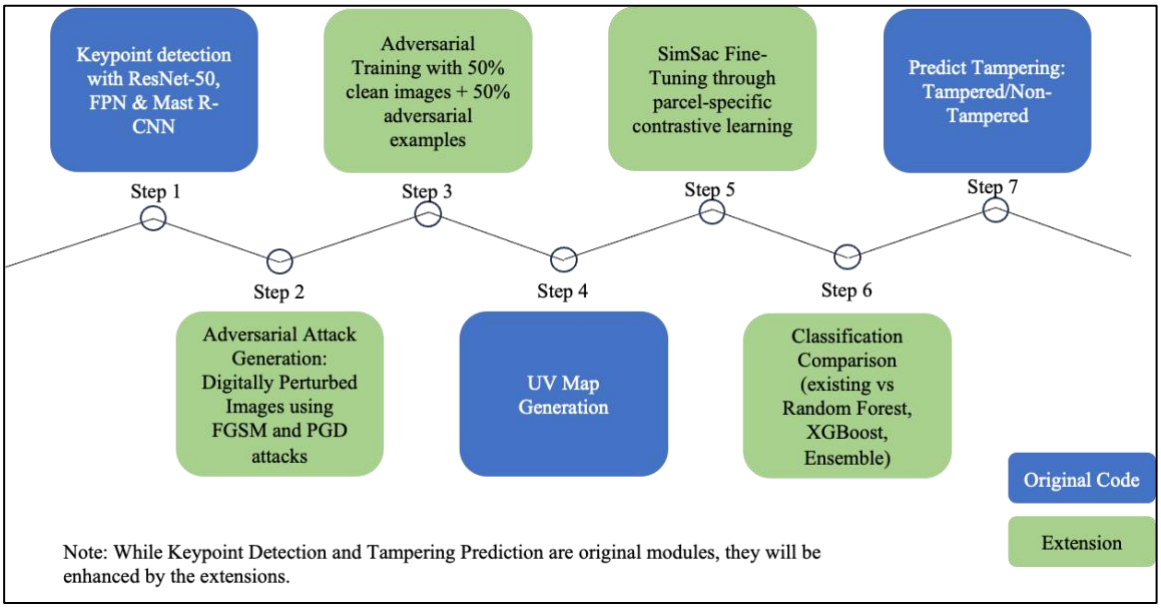

Figure 2: Pipeline overview illustrating Original Code and proposed Extensions in the pipeline

Step 1 which is Keypoint detection is part of the original TAMPAR code. In this step key points are detected using Mask R-CNN with ResNet-50 & FPN for 8-keypoint detection.

Extension A is split into two steps namely step 2 and 3. Step 2 is Adversarial attack generation. Here the proposed work will implement **FGSM** and **PGD** attacks to generate adversarial examples with varying perturbation budgets and further measure attack success via keypoint displacement error and detection failure rate. Once baseline is established pipeline proceeds with step 3 i.e. Adversarial Training. In this step, the model is trained using mixed batches of clean & adversarial samples to improve its robustness. This work will focus on having 50% clean and 50% adversarial images as inputs to training batches. This step will also involve fine tuning of TAMPAR pre-trained weights to improve its robustness.

Step 4 of geometric transformations using predicted keypoints a.k.a UV Map generation is unchanged. Here the functionality of UV Map generation is not being changed instead the same technique is being used to generate UV Maps for adversarial examples.

Extension B is further split into two steps namely step 5 and 6. In the original pipeline the similarity scores were generated using SimSaC for semantic correspondence and change detection with similarity metrics: MS-SSIM, CW-SSIM, SSIM, HOG, MAE. Here the SimSaC used was not fine-tuned for parcel-specific images. Hence in this extension, as part of step 5, Transfer learning will be implemented on SimSaC for parcel-specific image pairs. For this purpose Positive & Negative pairs will be used for contrastive learning. This is illustrated using Table 1. This transfer learning will be done in two phases based on the bandwidth. In the first phase, the proposed work will keep the backbone frozen and train projection head only. If time permits and compute limitations are not hit, the work will be extended with second phase i.e. unfreeze multiple layers of the backbone to further fine-tune the model.

Pair Type	Surface 1	Surface 2	Label	Purpose
Positive Pairs				
Clean Reference Vs Clean uvmap_pred (All Angles)	Parcel A, Top Surface, View 1	Parcel A, Top Surface, View 2	1 (Match)	Learn viewpoint invariance per surface

Clean Reference Vs Clean uvmap_gt (All Angles)	Parcel A, Front Surface (Reference UV)	Parcel A, Front Surface (Predicted UV)	1 (Match)	Match clean predictions surface-wise
Clean Reference Vs Augmented Clean Reference	Parcel A, Left Surface (Original)	Parcel A, Left Surface (Rotated/Brightened)	1 (Match)	Robustness to lighting/angle variations
Clean Reference Vs Adversarial Clean Surfaces	Parcel A, Left Surface (Original)	Parcel A, Left Surface (Adversarial)	1 (Match)	Robustness to adversarial noise
Negative Pairs				
Clean Reference Vs Tampered uvmap_gt	Parcel A, Front Surface (Clean)	Parcel A, Front Surface (Tampered)	0 (No Match)	Detect tampering on specific surfaces
Clean Reference Vs Tampered uvmap_pred	Parcel A, Front Surface (Clean)	Parcel A, Front Surface (Tampered)	0 (No Match)	Detect tampering on specific surfaces
Clean Reference Vs Adversarial Tampered Surfaces	Parcel A, Right Surface (Clean)	Parcel A, Right Surface (Adversarial)	0 (No Match)	Robustness to adversarial perturbations

Table 1 Contrastive Learning Pair Strategy (Surface-Level)

In step 6, the pipeline focuses on improving the classification. Currently the similarity scores are fed into decision tree to provide predictions. In this step, existing decision tree predictions will be compared with Random Forest, XGBoost and their ensembles.

In Step 7, the pipeline will incorporate the predictor with best results in the main pipeline to generate predictions for tampering classification.

Pipeline Setup:

The development environment was configured on GitHub Codespaces, provisioned with a 2-core CPU, 8 GB of RAM, and 32 GB of storage. This remote environment was accessed via VS Code on a local machine, which could remain lightweight since all development and debugging activities were performed remotely. Model training, validation, and testing were carried out on Google Colab using GPU acceleration. For most pipeline stages, a T4 instance with High-RAM configuration was sufficient; however, adversarial image generation required the more powerful A100 instance due to its higher memory demands. It should be noted that these infrastructure requirements are highly dependent on the size of the dataset used.

Distinct hyperparameter configurations were employed across different pipeline stages. For adversarial image generation, a perturbation budget of $\epsilon = 10$ was used. SimSac fine-tuning was performed with a batch size of 8 over 20 epochs. For classifier comparison, Random Forest was initialized with 100 estimators and a maximum depth of 8, while XGBoost was configured with 100 estimators, a maximum depth of 5, and a learning rate of 0.1. The ensemble model combined a Decision Tree, Random Forest, and XGBoost using the same individual hyperparameters stated above.

Hyperparameter tuning via grid search was conducted for both Random Forest and XGBoost. For Random Forest, the search space covered estimators $\in\{100,200,300\}$ and maximum depth $\in\{\text{None},5,10\}$. For XGBoost, estimators $\in\{200,300\}$, maximum depth $\in\{3,6\}$, and learning rate $\in\{0.05,0.1\}$ were explored.

Evaluation Metrics:

As verified in the code provided with the original paper, the results published in the paper are of the k-cross validation with k as 5 instead of test scores. As part of this project, first step would be to baseline the code to show test accuracy and scores prior to making any changes in the pipeline. Doing

so will ensure two things, firstly the results would reflect the actual test results in the new setup and secondly, all the changes are compared with actual test accuracy.

There would be more than one evaluation metric for various tasks. For evaluating robustness the proposed work would be using Attack success rate, robust accuracy, transferability analysis. For detection purpose the evaluation matrix will include Precision, Recall, F1-score, AUC-ROC. As part of this research, adversarial dataset will be released as a first comprehensive attack benchmark for parcel tampering systems.

Justification:

The choice of FGSM and PGD attacks provides a comprehensive evaluation spectrum from fast gradient-based attacks to optimization-based sophisticated attacks. Adversarial training is chosen as the primary defence as it has demonstrated effectiveness across various vision tasks. SimSaC fine-tuning leverages the power of contrastive learning which has shown superior performance in similarity learning tasks. Random forests are selected for their ability to handle non-linear relationships, feature importance ranking, and resistance to overfitting

7. Results:

We successfully completed base setup tasks including environment configuration (PyTorch, Detectron2 etc), TAMPAR dataset download and pre-trained weight initialization. Below are the baseline Keypoint Detection and Tampering Detection results from the original implementation.

	predictor	compare_types	scores	accuracy	precision_binary	recall_binary	f1_binary	roc_auc
0	simple_threshold	plain	mae, hog, cwssim, ssim, msssim	0.6808	0.6856	0.9189	0.7853	0.5927
1	simple_threshold	simsac	mae, hog, cwssim, ssim, msssim	0.8385	0.9805	0.7609	0.8566	0.8672
2	simple_threshold	canny	mae, hog, cwssim, ssim, msssim	0.6783	0.6705	0.9706	0.7931	0.5700
3	simple_threshold	laplacian	mae, hog, cwssim, ssim, msssim	0.6551	0.7024	0.8002	0.7445	0.6009
4	simple_threshold	meanchannel	mae, hog, cwssim, ssim, msssim	0.6795	0.6877	0.9128	0.7813	0.5929
5	simple_threshold	plain, simsac, canny, laplacian, meanchannel	mae, hog, cwssim, ssim, msssim	0.8340	0.9815	0.7528	0.8520	0.8640

Table 2 Baseline Simple Threshold Tampering Detection results

FGSM and PGD adversarial attacks have been successfully implemented and adversarial datasets generated. An equal number (439) of adversarial images have been generated for each attack type viz. PGD and FGSM collectively for test and validation datasets. Some of the homogenization methods have been used to visualize how clean and adversarial images are viewed by these methods. Adversarial surface images clearly show the added noise to the original image.

Figure 3: Example of center surface for a parcel which was tampered on field with a label and how reference(R), clean (C) and adversarial (A) versions are viewed by image homogenization method Canny.

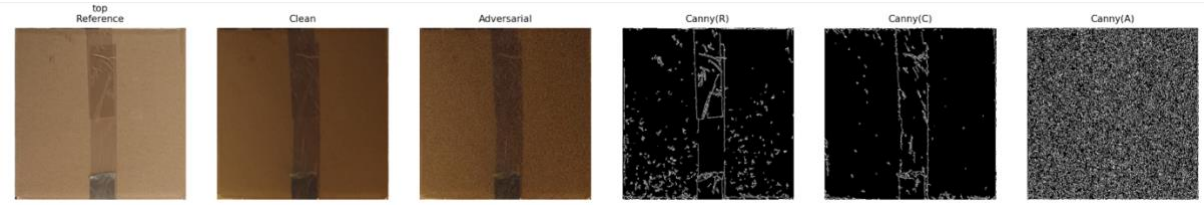

Figure 4: Example of center surface for a parcel which was not tampered on field and how reference(R), clean (C) and adversarial (A) versions are viewed by image homogenization method Canny.

Figure 5: Example of right surface for a parcel which was tampered on field with writing and how reference(R), clean (C) and adversarial (A) versions are viewed by image homogenization method SimSAC.

These attacks reveal a significant vulnerability in the tampering detection system, with accuracy dropping by ~10 percentage points (from ~82% to ~72%) with adversarial inputs.

	predictor	compare_types	scores	accuracy	precision_binary	recall_binary	f1_binary	roc_auc
0	simple_threshold	plain	hog, cwssim, msssim, ssim, mae	0.6750	0.6892	0.8914	0.7770	0.5944
1	simple_threshold	simsac	hog, cwssim, msssim, ssim, mae	0.7221	0.9535	0.5922	0.7303	0.7705
2	simple_threshold	canny	hog, cwssim, msssim, ssim, mae	0.6268	0.6611	0.8641	0.7429	0.5893
3	simple_threshold	laplacian	hog, cwssim, msssim, ssim, mae	0.6674	0.6744	0.9228	0.7790	0.5723
4	simple_threshold	meanchannel	hog, cwssim, msssim, ssim, mae	0.6697	0.6943	0.8620	0.7673	0.5981
5	simple_threshold	plain, simsac, canny, laplacian, meanchannel	hog, cwssim, msssim, ssim, mae	0.7169	0.9586	0.5798	0.7221	0.7679

Table 3 Simple Threshold Tampering Detection results on Adversarial images

On the other hand, key-point detection remains robust ~97.89 on clean images and 97.70 on adversarial images (Table 4). Originally planned Tasks #3.3 to #3.7, #7.1 & #7.2 are no longer applicable, as Keypoint Detection is robust to adversarial attacks, eliminating the need for retraining and improvement.

```
Task: bbox
AP,AP50,AP75,APs,APm,APl
95.9046,100.0000,100.0000,nan,nan,95.9046
Task: keypoints
AP,AP50,AP75,APm,APl
97.8934,100.0000,100.0000,nan,97.8934
Task: segm
AP,AP50,AP75,APs,APm,APl
98.9462,100.0000,100.0000,nan,nan,98.9462
```

```
Task: bbox
AP,AP50,AP75,APs,APm,APl
95.9617,100.0000,100.0000,nan,nan,95.9617
Task: keypoints
AP,AP50,AP75,APm,APl
97.7061,100.0000,100.0000,nan,97.7061
Task: segm
AP,AP50,AP75,APs,APm,APl
98.9959,100.0000,100.0000,nan,nan,98.9959
```

Table 4 Baseline Keypoint Detection results (left) and Kepoint Detection results on Adversarial images (right)

We successfully fine-tuned the SimSAC change detection model using a two-phase contrastive learning approach to improve tampering detection robustness using the contrastive pair strategy as defined in Table 1. And Table 5 displays the statistics of the Positive and Negative Pairs created. Phase 1 employed supervised fine-tuning with a frozen VGG backbone, while Phase 2 performed full model fine-tuning with contrastive loss on surface-level positive/negative pairs.

Pair Creation Statistics		
Positive Pairs		
	Reference vs uvmap_pred (clean)	361
	Reference vs uvmap_gt (clean)	369
	Reference vs Augmented Reference	114
	Reference vs Adversarial (clean)	695
	Total Positive	**1,539**
Negative Pairs		
	Reference vs Tampered uvmap_gt	561
	Reference vs Tampered uvmap_pred	551
	Reference vs Adversarial (tampered)	1,066
	Total Negative	**2,178**
Grand Total		**3,717**

Table 5 Pair Creation Statistics (Surface-Level)

The training and validation loss curves demonstrate effective convergence during SimSAC fine-tuning. Both losses drop sharply within the first two epochs, indicating rapid initial learning of contrastive representations. The training loss continues to decrease steadily. Validation loss closely tracks training loss throughout, suggesting the model generalizes well without overfitting. The minimal gap between the two curves indicates appropriate model capacity and regularization. Overall, the smooth convergence pattern confirms successful fine-tuning of the SimSAC contrastive learning framework.

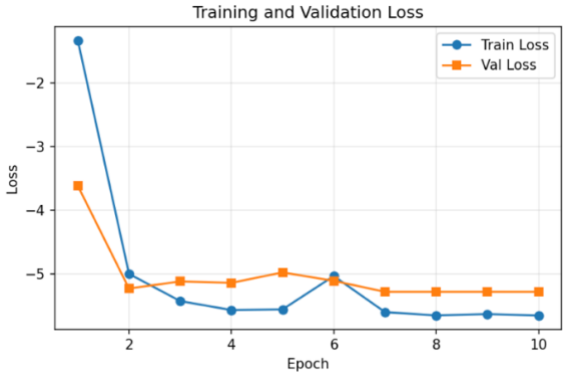

Figure 6 SimSAC fine-tuning Training and Validation Loss for Contrastive Learning

The fine-tuned model achieved significantly better separation between positive and negative pairs (Figure 7) compared to the baseline pre-trained model (Figure 8). With baseline pre-trained model, there is a significant overlap in positive and negative surface pairs for adversarial surface images.

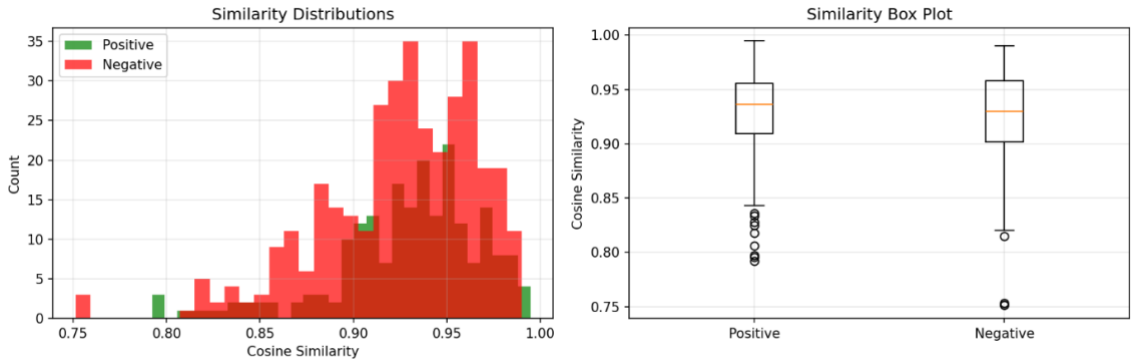

Figure 7 Similarity Distributions for surface-wise pairs with baseline pre-trained model

Post fine-tuning, positive pairs tightly clustered near 1.0 (median ~0.98) while negative pairs distributed broadly from 0.30-0.70 (median ~0.55), demonstrating substantial improvement in discriminative capacity.

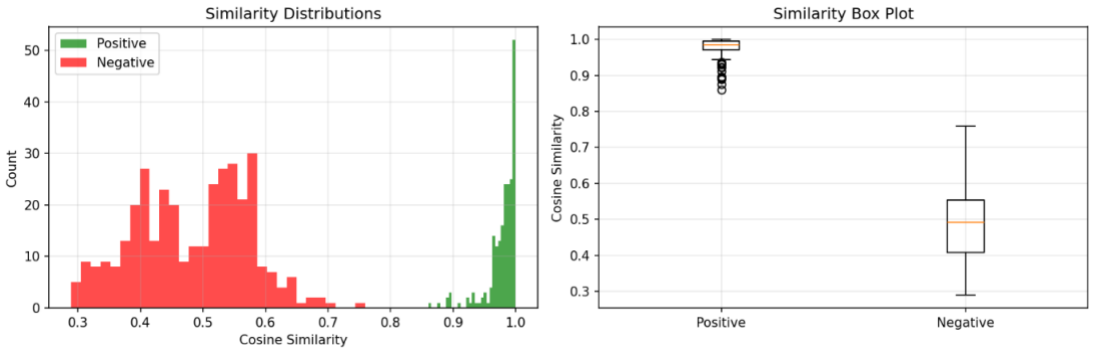

Figure 8 Similarity Distributions for surface-wise pairs with fine-tuned model

Above fine-tuning approach used contrastive learning, where the model was trained to push feature embeddings of tampered image pairs apart and pull clean pairs together in a high-dimensional vector space. Visualising the resulting cosine similarity scores showed a clear separation between tampered and clean pairs, which initially appeared promising.

However, this visual result was misleading. SimSAC does not use these similarity embeddings at inference time. It uses change maps: per-pixel difference maps computed from intermediate feature layers. The contrastive projection head, which produced the well-separated embeddings, is discarded during detection. As a result, training the projection head had no direct effect on the change maps that the tampering detector actually relies on. Improved embedding separation did not translate into improved tampering detection accuracy.

To address this, a new training objective was designed that directly optimises the change maps used during detection. Rather than training an auxiliary projection head, the loss function acts on the change map values themselves:

For a tampered pair: the mean change map value should be high (the model should "see" a difference)
For a clean pair: the mean change map value should be low (the model should see no difference)
The training objective can be expressed simply as:

$$Loss = (1 - 2 \times label) \times mean\ change\ map \qquad (1)$$

where label = 1 for tampered pairs and 0 for clean pairs. This formulation pushes the tampered change map up and the clean change map down in a single, unified term.

To ensure gradients reach all parts of the model, both the VGG feature pyramid (backbone) and the decoder layers are unfrozen during training with the backbone trained at a lower learning rate to preserve its general-purpose features while still allowing adaptation to adversarial conditions.

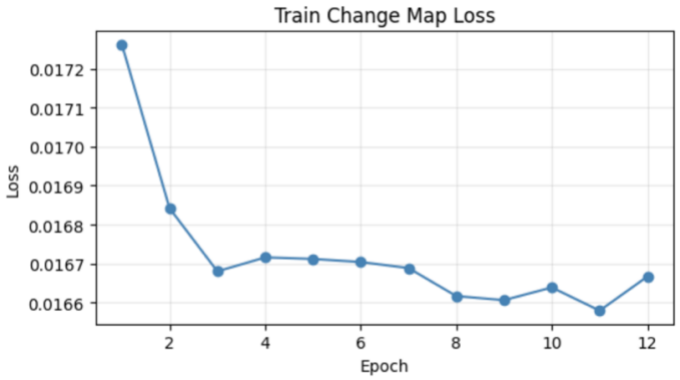

Figure 9 Training loss curve for the direct change map fine-tuning of SimSAC

The training loss curve above shows the direct change map loss decreasing steadily over 12 epochs, confirming that the model is successfully learning to produce higher change maps for tampered pairs and lower change maps for clean pairs. The consistent downward trend demonstrates that direct change map supervision provides a clear and stable learning signal.

Three classifiers viz. Random Forest, XGBoost, and a soft-voting Ensemble were evaluated on similarity score features extracted by the fine-tuned SimSAC model under two conditions: clean images and adversarially perturbed images. As shown in Table 6, XGBoost consistently outperforms the other classifiers across both conditions.

Compared to the original SimSAC weights, the fine-tuned model shows clear improvements across both conditions. On clean images, XGBoost achieves 87.00% accuracy and a ROC-AUC of 0.9436, up from a baseline of 83.85% accuracy and 0.8672 ROC-AUC. The gains are far more pronounced on adversarial images, where accuracy improves from 72.21% to 85.99% and ROC-AUC rises from 0.7705 to 0.9385, an improvement of over 13 and 16 percentage points respectively. Recall on adversarial images also improves dramatically, from 59.22% to 95.32%, indicating that the fine-tuned model combined with XGBoost classifier is substantially better at catching tampered parcels even under adversarial attack.

	predictor	compare_types	scores	accuracy	precision_binary	recall_binary	f1_binary	roc_auc
1	random_forest	simsac	cwssim, ssim, msssim, hog, mae	0.8591	0.9666	0.8065	0.8788	0.9398
7	xgboost	simsac	cwssim, ssim, msssim, hog, mae	0.8700	0.9855	0.8075	0.8872	0.9436
13	ensemble	simsac	cwssim, ssim, msssim, hog, mae	0.8552	0.9778	0.7903	0.8734	0.9379

	predictor	compare_types	scores	accuracy	precision_binary	recall_binary	f1_binary	roc_auc
1	random_forest	simsac	mae, msssim, hog, cwssim, ssim	0.8236	0.8246	0.9176	0.8686	0.9061
7	xgboost	simsac	mae, msssim, hog, cwssim, ssim	0.8599	0.8461	0.9532	0.8964	0.9385
13	ensemble	simsac	mae, msssim, hog, cwssim, ssim	0.8265	0.8234	0.9259	0.8716	0.9099

Table 6: Performance Comparison of Classifiers over SimSAC Fine-tuned Features (top: clean images, bottom: adversarial images), with XGBoost yielding the best result across both conditions.

XGBoost's superior performance can be attributed to its gradient boosting mechanism, which iteratively corrects residual errors and handles the non-linear relationships in the similarity score feature space more effectively than a single ensemble vote. The accuracy gain over the baseline demonstrates that fine-tuning SimSAC with direct change map supervision, combined with XGBoost classification, produces a measurably more robust tampering detection pipeline against adversarial attacks.

Qualitative Analysis: Model Predictions on Adversarial Examples:

Figure 10 presents four representative examples from the adversarial test set, illustrating the model's ability to correctly classify both tampered and clean parcel surfaces under adversarial conditions.

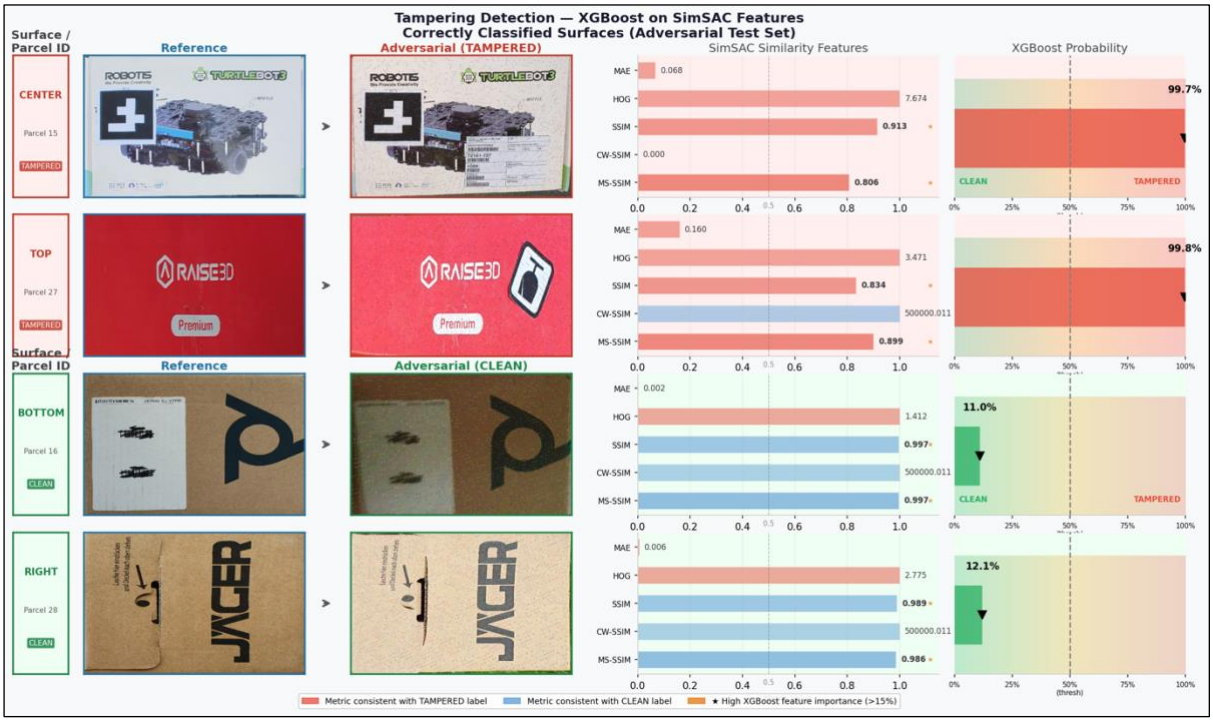

Figure 10 Tampering Detection Using XGBoost on SimSaC Features - Representative Classification Results on Adversarial Test Set

For each example, the figure displays the reference image alongside its adversarial counterpart, the SimSaC similarity feature scores, and the corresponding XGBoost tampering probability.

The first two examples viz. Parcel 15 (CENTER surface) and Parcel 27 (TOP surface) are genuinely tampered parcels. Despite adversarial perturbations applied to the input, the model correctly identifies both as tampered with a high confidence of 99.8%. The SimSAC features for these parcels show elevated HOG dissimilarity and relatively lower SSIM and MS-SSIM scores, which are consistent with tampered surface characteristics and align with the features identified as high importance by XGBoost.

The third and fourth examples viz. Parcel 23 (LEFT surface) and Parcel 16 (CENTER surface) are clean parcels. The model correctly classifies both as untampered, assigning low tampering probability scores of 9.9% and 11.0% respectively. Their SimSAC features reflect high structural similarity with SSIM and MS-SSIM values approaching 1.0, consistent with clean surface characteristics.

These results demonstrate that the model maintains reliable classification performance on both tampered and clean samples even when evaluated on adversarially perturbed inputs, correctly identifying all four examples with high confidence in the expected direction.

8. Implementation and User Benefit:

The enhanced TAMPAR system addresses critical security vulnerabilities in last-mile delivery logistics. Extension A protects against adversarial

manipulation where malicious actors could add imperceptible digital perturbations to conceal tampering evidence. Extension B reduces false positives through improved feature representations and advanced classification, enhancing customer trust and operational efficiency where manual verification is impractical across millions of daily deliveries. The system provides real-time tampering detection at scale, enabling logistics providers to maintain supply chain integrity while minimizing human inspection costs. The adversarial benchmark dataset will establish industry-standard security evaluation protocols for parcel verification systems.

9. Limitations and Further Improvements:

Future improvements could explore additional adversarial attack types (C&W attacks), investigate defensive distillation and certified defence mechanisms, extend to physical adversarial patches beyond digital perturbations, incorporate multi-modal features (thermal imaging, weight sensors), and scale to larger datasets with more parcel variations and tampering categories.

10. Bibliography and References:

[1] Naumann, S., et al. (2024). TAMPAR: Visual Tampering Detection for Parcels in Postal Supply Chains. 2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2024 [Online]. Available: https://arxiv.org/abs/2311.03124

[2] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards Deep Learning Models Resistant to Adversarial Attacks," Sep. 2019, [Online]. Available: http://arxiv.org/abs/1706.06083

[3] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," Jul. 2020, [Online]. Available: http://arxiv.org/abs/2002.05709

[4] S. Thys, W. Van Ranst, and T. Goedemé, "Fooling automated surveillance cameras: adversarial patches to attack person detection." [Online]. Available: https://github.com/pjreddie/darknet/

[5] TAMPAR Dataset. (2023). Zenodo. https://zenodo.org/records/10057090

11. Individual Progress:

Task	Sub-task	Owner	ClickUp Task ID	Status
Task 1: Environment Setup & Baseline	1.1: Download TAMPAR dataset	All Members	86d18y5fz	**Complete**
	1.2: Set up development environment (PyTorch, Detectron2 and other dependencies)	All Members	86d18y5g3	**Complete**
	1.3: Download pre-trained TAMPAR keypoint detection weights	All Members	86d18y5g5	**Complete**
	1.4: Run baseline keypoint detection and document metrics	All Members	86d18y5g6	**Complete**
	1.5: Generate ground truth UV maps using annotations	All Members	86d18y5g7	**Complete**
	1.6: Run baseline tampering detection pipeline (SimSaC + decision tree)	All Members	86d18y5gb	**Complete**
	1.7: Document baseline performance (keypoint AP, tampering F1-score)	All Members	86d18y5ge	**Complete**

	1.8: NEW- modify code to be compatible with environment available in google colab	All Members	86d1qaa56	**Complete**
	1.9: NEW- Modify code to be compatible for CPU so that GIT codespace can be used for rapid development and debugging	Nishith Mohan Mittal	86d1wfc35	**Complete**
Task 2: Adversarial Attack Implementation (Extension A)	2.1: Implement FGSM attack on keypoint detector	Pallav Krishna Rakesh	86d18y5gg	**Complete**
	2.2: Test FGSM with epsilon values	Nishith Mohan Mittal	86d18y5gm	**Complete**
	2.3: Implement PGD attack with iterations	Pallav Krishna Rakesh	86d18y5gp	**Complete**
	2.4: Generate adversarial dataset	Pallav Krishna Rakesh	86d18y5gu	**Complete**
	2.5: Compute attack success metrics	Nishith Mohan Mittal	86d18y5gw	**Complete**
	2.6: Visualize attack examples and document results	Nishith Mohan Mittal	86d18y5gy	**Complete**
Task 3: Adversarial Training (Extension A)	3.1: Prepare adversarial training dataset (50% clean + 50% adversarial)	Pallav Krishna Rakesh	86d18y5h1	**Complete**
	3.2: Implement adversarial training loop with PGD augmentation	Pallav Krishna Rakesh	86d18y5h4	**Complete**
Task 4: UV Map Generation for SimSaC (Extension B)	4.1: Generate predicted UV maps using baseline keypoint detector	Rakesh Sharma	86d18y5hq	**Complete**
	4.2: Extract surface patches (top, left, right, center, bottom) from UV maps	Rakesh Sharma	86d18y5hv	**Complete**
	4.3: Create positive pairs (as per positive pair strategy in Table 1)	Rakesh Sharma	86d18y5hy	**Complete**
	4.4: Create negative pairs (as per negative pair strategy in Table 1)	Rakesh Sharma	86d18y5j3	**Complete**
	4.5: Implement data augmentation	Rakesh Sharma	86d18y5j5	**Complete**
	4.6: Prepare final training dataset	Rakesh Sharma	86d18y5j8	**Complete**
Task 5: SimSaC Fine-tuning (Extension B)	5.1: Set up SimSaC architecture and load pre-trained weights	Rakesh Sharma	86d18y5jb	**Complete**
	5.2: Implement combined loss	Rakesh Sharma	86d18y5je	**Complete**
	5.3: Implement training loop (Phase 1, freeze SimSaC feature extractor, train projection head only)	Rakesh Sharma	86d18y5jg	**Complete**

	5.4: Train SimSaC Phase 1	Rakesh Sharma	86d18y5jj	**Complete**
	5.5: Implement fine-tuning loop (Phase 2, unfreeze all layers)	Rakesh Sharma	86d18y5jp	**Complete**
	5.6: Train SimSaC Phase 2	Rakesh Sharma	86d18y5jq	**Complete**
	5.7: Evaluate fine-tuned SimSaC on validation set	Rakesh Sharma	86d18y5jt	**Complete**
	5.8: Extract improved similarity features for classification	Rakesh Sharma	86d18y5jx	**Complete**
Task 6: Enhanced Classification (Extension B)	6.1: Extract fine-tuned SimSaC features from validation set	Rakesh Sharma	86d18y5jz	**Complete**
	6.2: Apply SMOTE for class balancing on training set	Nishith Mohan Mittal	86d18y5k2	**Complete**
	6.3: Implement Random Forest classifier	Nishith Mohan Mittal	86d18y5k4	**Complete**
	6.4: Implement XGBoost classifier	Yashwanth Maringanti	86d18y5k6	**Complete**
	6.5: Implement soft voting ensemble (Random Forest + XGBoost)	Nishith Mohan Mittal	86d18y5k8	**Complete**
	6.6: Hyperparameter tuning using grid search on validation set	Nishith Mohan Mittal	86d18y5kb	**Complete**
	6.7: Evaluate on test set and document improvements (F1-score, AUC-ROC)	Nishith Mohan Mittal	86d18y5kf	**Complete**
	6.8: Perform per-surface analysis (top, left, right, center, bottom)	Yashwanth Maringanti	86d18y5kh	**Complete**
Task 7: Integration & End-to-End Testing	7.3: Integrate fine-tuned SimSaC into pipeline	Nishith Mohan Mittal	86d18y5kt	**Complete**
	7.4: Integrate enhanced classifier (ensemble) into pipeline	Nishith Mohan Mittal	86d18y5kw	**Complete**
	7.5: Run end-to-end pipeline on full test set	All Members	86d18y5kz	**Complete**
	7.6: Compare baseline vs. enhanced system performance	All Members	86d18y5m1	**Complete**
	7.7: Perform ablation studies (SimSaC only, RF only, combined)	All Members	86d18y5m2	**Complete**
Task 8: Benchmark & Documentation	8.1: Prepare adversarial benchmark dataset for release	Pallav Krishna Rakesh	86d18y5mc	**Complete**
	8.2: Write evaluation scripts for adversarial robustness testing	Nishith Mohan Mittal	86d18y5mf	**Complete**
	8.3: Document all hyperparameters and training procedures	Nishith Mohan Mittal	86d18y5mn	**Complete**

	8.4: Create architecture diagrams (original + enhancements)	Rakesh Sharma	86d18y5mq	**Complete**
	8.5: Generate result visualizations (attack success curves, confusion matrices)	Yashwanth Maringanti	86d18y5mu	**Complete**
	8.6: Write final project report (methodology, results, analysis)	All Members	86d18y5mw	**Complete**

Table 7 Individual progress tracking with ClickUp task IDs

Note: Tasks 3.3–3.7, 7.1, and 7.2 were removed from the task list as keypoint detection remained robust under adversarial conditions (97.89% clean vs. 97.70% adversarial), making adversarial retraining of the keypoint detector unnecessary.

12. Appendix:

Hardware Configuration:

The primary training environment was Google Colab Pro+, utilizing NVIDIA T4 (16GB VRAM) and A100 (40GB VRAM) GPUs depending on task requirements. High-memory A100 instances were used for adversarial training and SimSaC fine-tuning, which involve batch processing of high-resolution (4032×3024) images. T4 instances were used for feature extraction and classifier training. Colab Pro+ provided sufficient compute, storage, and GPU memory for all experimental tasks without requiring dedicated local infrastructure.

Software Platform & Tools:

The implementation leverages PyTorch with CUDA for model training and adversarial attack generation, Detectron2 for Mask R-CNN keypoint detection, and Torchvision for data augmentation pipelines. Adversarial attacks (FGSM, PGD, and C&W) are implemented directly in PyTorch without external attack libraries. Machine learning components utilize Scikit-learn for Random Forest classification and evaluation metrics, alongside XGBoost for gradient boosting classification. NumPy and Pandas handle data manipulation and feature extraction, while OpenCV provides image I/O operations, geometric transformations, and UV map generation functionality.